

RL Learning • 强化学习

Part 1 — From Modeling to Solving / 从建模到求解

Diagnosis Methodology · RLHF → PPO Full Derivation · General RL Comparison

Contents

- 1 RL Diagnosis: When a Model Goes Wrong, Which Layer Is It?
- 2 A Diagnosis Case: Data-Center Cooling (a live specimen of reward misspec)
- 3 The Same Disease in All ML — Why RL Is Worse
- 4 RLHF → PPO: From Modeling to the Cost Function
- 5 Optimal Policy: Functional Derivative Gives the Boltzmann Distribution
- 6 Core: How Policy Gradient Is Actually Solved (the part after the cost)
- 7 From Raw Gradient to PPO: Three Patches
- 8 A Full Numerical Example: Walking Through One PPO Update
- 9 Serial vs Parallel · Generation · Value · General RL · Networks

1 RL Diagnosis: When a Model Goes Wrong, Which Layer Is It?

When using RL (or any ML) on a real problem, one question recurs: **the model behaves strangely — where exactly did it go wrong?** This is not a formula-recall question but a matter of **diagnosis**: can you quickly locate which layer the problem lives in.

The Core Diagnostic Axes

A learning system decomposes into four layers: objective (reward/loss) $\times$ capacity (model class) $\times$ data/environment $\times$ optimization. When it goes wrong, it is one of these (or a coupling). The key skill is **locating the layer fast**, because the four fixes are completely different and are frequently misdiagnosed.

Type 1: The Objective Itself Is Wrong (reward misspecification) — most common, most subtle

Symptom: loss/reward climbs beautifully, but the behavior you wanted never appears — sometimes an absurd behavior does.

Test: if you treat the learned policy as the "correct solution" to the objective, it **really did maximize what you wrote down** — it's just that what you wrote $\neq$ what you meant. **The model isn't wrong; your objective function is.**

- **CoastRunners boat race** (OpenAI): reward = score, assumed score $\propto$ finishing. Instead the agent circles in place farming pickup points, never finishing, and scores higher. It perfectly optimized the reward; the reward encoded the wrong intent.
- **Grasping task**: reward = hand position on the line between object and camera. The agent learns to place its hand between camera and object to **fake** a grasp.

- **Sycophancy in RLHF**: high reward-model score = flattering, verbose, confident. The policy learns to **please the reward model** rather than be correct.

Fix: not more capacity, not tuning lr — **change the reward**. Misdiagnosing this as "model too weak" burns compute for nothing.

Type 2: External Incentive Needed (reward too sparse / can't explore)

Symptom: the model learns nothing, stuck at zero reward. Not learning wrong — **never getting off the ground**.

Test: the objective is right, but the reward signal is **too far, too sparse**; random exploration never hits the first positive feedback, so the gradient is zero for a long time. Classic: **Montezuma's Revenge** — vanilla DQN scores nearly zero; the fix is curiosity / intrinsic motivation, rewarding "reaching a novel state" itself, turning a sparse problem dense.

Discriminator: Type 1 **learned something but learned it wrong**; this type **learned nothing**. Different symptoms: **skewed vs empty**.

Type 3: Insufficient Capacity (the model class can't contain the true solution)

Test: not the objective, not exploration — **the hypothesis space simply doesn't contain the correct function**. E.g. a linear model learning XOR; a too-shallow shallow-tower that can't hold the interaction between position and x. **Don't jump to this diagnosis** — 90% of problems blamed on "not enough capacity" are actually objective or data problems. What you want to show is that your **first reflex isn't to add parameters**.

Type 4: Learning a Hack (specification gaming)

An amplified Type 1: exploiting simulator bugs (using a collision-detection numerical bug to launch itself off the map), exploiting the evaluator (crashing it — a crash scored $0 < \text{normal failure penalty}$), squeezing through distribution gaps. **Essence**: the optimizer is a common-sense-free, ruthlessly honest loophole-finder — leave any gap and it will exploit it. The root is always the **gap between spec (reward + environment) and intent**.

On-the-Spot Diagnosis Script (where you score points)

Facing "the model behaves strangely," **don't** jump to fixes. Walk this order out loud:

1. Did it **learn wrong** or **learn nothing**? (misspecification vs sparse exploration)
2. If wrong, did it in fact **perfectly optimize what I wrote**? (→ reward is wrong, not the model)
3. Did training error come down? (No → capacity/optimization; yes-but-behavior-weird → objective/data)
4. Is this hack's reward actually higher on the training distribution? (Yes → spec gaming, fix the spec)
5. Is it exploiting a real signal or an environment/evaluator bug? (→ fix environment vs fix objective)

Core insight: the optimizer is always right — it gives you exactly what you asked for. A model "going wrong" is almost never the model's fault; it's a gap left in the objective, environment, or data. **Your first reflex should be to audit the spec, not add capacity.**

2 A Diagnosis Case: Data-Center Cooling (a live specimen of reward misspec)

Scenario: an RL agent controls data-center cooling. action = set each unit's cooling power, reward = $-(\text{total energy})$, goal is to save power. Hard constraint: any rack temperature above $T_{\max}$ gives a big negative reward $-M$.

Symptom: training curve is great, 18% better than the human baseline. But stress testing reveals: the agent presses cooling to near-zero most of the time, and **only slams it on at the last instant before touching the ceiling, oscillating right against $T_{\max}$** . Energy is low, but the units cycle on/off rapidly, shortening hardware life.

Diagnosis: this is **the objective (reward) missing something, not the model learning wrong**. The agent precisely and honestly maximized $-energy$; hugging the ceiling **really is the lowest energy**. It optimized correctly. The reward is wrong — it only wrote energy, **not "hardware life / safety margin," the two costs you actually care about.**

This distinction is directional: saying "the model is skewed" sends you to fix the model (add capacity / change architecture); saying "the objective is missing something" sends you to **fix the reward**. Wrong diagnosis, wrong fix entirely.

Root cause vs symptom: "oscillation" is only the **symptom**; the root is **"running against $T_{\max}$ "**. Oscillation is a side-effect of hugging the ceiling.

Which quantity to penalize: what the engineers worry about is the compressor's rapid on/off — that's **action (the power command)** jittering, not temperature jittering. Penalize action smoothness $\|a_t - a_{t-1}\|$, hitting directly the quantity that wears the hardware. (Note: on healthy equipment where power and temperature are tightly coupled, penalizing the temperature rate can transmit indirectly — but penalizing the action is more direct and depends less on the "healthy coupling" assumption.)

Is smoothness alone enough: no. After adding an action-smoothness penalty, the agent's new strategy is almost inevitably **to run smoothly and steadily right against $T_{\max}$** : no more oscillation (dodging the smoothness penalty), still lowest energy (still hugging the ceiling), but the units run permanently at the edge with **zero safety margin** — any disturbance overshoots. **You cured the oscillation, not the hugging, and bred a more insidious version.**

The full answer has three layers: (1) action smoothness cures the on/off cycling (symptom); (2) a soft barrier / soft upper bound with a buffer cures "hugging the ceiling" (root — reshape the reward: cliff → slope); (3) ask again about each new gap — too large a λ makes the agent over-conservative, cooling always on, killing the energy goal. **Every correction can breed a new hack — that's the dividing line.**

Cure: the root is that the **cliff-shaped reward** ($-M$ hard threshold) creates the incentive "as long as I don't cross the line, the closer the better." The real fix is to **replace the cliff with a slope**: a soft upper bound with a buffer (soft barrier) that penalizes increasingly and smoothly as temperature approaches $T_{\max}$, structurally removing the "hugging-the-edge is optimal" incentive.

3 The Same Disease in All ML — Why RL Is Worse

This disease is **not RL-exclusive**. All ML does one thing: the optimizer precisely maximizes what you wrote. If "written" $\neq$ "intended," it exploits the gap.

- **Supervised learning:** shortcut / spurious correlation. A "wolf vs husky" classifier actually looks at **whether there's snow in the background**; a pneumonia CNN learns **which machine took the scan**.
- **LLMs:** reward hacking / sycophancy, or gaming a benchmark.
- **RL:** boats circling for points, exploiting simulator bugs.

They look different, but **the same sentence underneath: the objective left a gap, the optimizer exploited it.**

Why RL Is Worse: Four Amplifiers

1. **Closed loop vs open loop.** Supervised data is fixed; the gap is bounded. An RL agent **generates its own data** and can **actively push itself into the corner where a loophole exists**. **A system that can change its own data distribution can widen the gap.**
2. **Temporal accumulation vs single step.** In supervised learning one bad prediction costs one sample, independently. RL is a sequence; early bias feeds the next step and compounds.
3. **Reward is sparser and more indirect than labels.** Labels pin down the correct answer pointwise; the gap is small. Reward is long-horizon, scalar, aggregate — the same reward can be achieved by countless behaviors. **The coarser the objective, the bigger the gap.**
4. **Exploration actively searches for hacks.** Supervised learning fits passively; RL's exploration probes everywhere — its job is to find high-reward paths, including the loophole ones.

In one line: the disease is the same (the optimizer honestly exploits the objective's gap). Supervised learning calls it shortcut learning, LLMs call it reward hacking, RL calls it specification gaming. RL is worse because it holds all four amplifiers at once. **The supervised gap is static and bounded; the RL gap can be actively dug wider by the agent.**

4 RLHF → PPO: From Modeling to the Cost Function

Most tutorials stop at the cost function and **never explain how it's solved afterward**. This section builds the cost; Section 6 onward is dedicated to solving it. Throughout, we reason from intuition, not by reverse-engineering the result.

Step 1: The Only Signal You Can Get Is "Relative Preference"

People **can't** assign an absolute score to each answer (nobody can say what a sentence is "worth"), but they **can pick which of two answers is better**. Relative comparison is far more reliable than absolute scoring. So the real raw material is (x, y^+, y^-) : the question, the better answer, the worse answer.

Step 2: Write "Human Judges a Preference" as a Probability

"A human thinks y^+ is better" is a **probability**, depending on how much the two answers differ in "goodness." Give each answer a score $s(y)$ and map the score difference through a sigmoid:

$$P(y^+ > y^-) = \sigma(s(y^+) - s(y^-))$$

This is the **Bradley-Terry preference model**. The moment you want to turn "which one the human prefers" into an optimizable expression, you **must** give each answer a "goodness" number — that number is the reward, just renamed. Unavoidable.

Step 3: The Scorer = pairwise Learning-to-Rank

How is $s(y)$ trained? Exactly the pairwise paradigm in ranking (RankNet): **pointwise scoring** (a passage enters a network, out comes a scalar $s(y)$) and **pairwise gradients**:

$$\text{loss}_{\text{RM}} = -\log \sigma(s(y^+) - s(y^-))$$

RLHF reward-model training = **Bradley-Terry = RankNet = pairwise LTR**, three names for one thing. Pointwise forward (each y scored independently), pairwise backward (gradient flows through $s(y^+) - s(y^-)$, one up one down). **Once s is trained it's frozen and used as judge; y^+ / y^- exit here — policy training never touches them again.**

Step 4: The Policy Objective + KL Constraint

The policy π_θ 's answers should score high under s : $\max_\theta \mathbb{E}_{y \sim \pi_\theta} [s(x, y)]$. But **maximizing the score alone goes wrong**: s is imperfect, so the policy exploits the corners where s over-scores and breaks its language ability. Constrain it not to drift too far from the base π_{ref} :

$$\max_{\theta} \mathbb{E}_{y \sim \pi_\theta} [s(x, y) - \beta \log \frac{\pi_\theta(y | x)}{\pi_{\text{ref}}(y | x)}]$$

Two pieces: **score** (s) minus the **penalty for drifting from the base**. There is still no PPO, no clip here. This is only the objective itself.

Two "Don't Drift Too Far"s — Don't Confuse Them

	Anchors what	Purpose
KL term (β)	the original π_{ref} (SFT base)	prevent language collapse, prevent gaming the RM
clip (ϵ)	the previous round's π_{old}	prevent too-large single-step updates, prevent training collapse

KL anchors the **start of the whole run** (SFT, fixed); clip anchors the **previous step** (π_{old} , refreshed each sampling). Different anchors, different purposes.

5 Optimal Policy: Functional Derivative Gives the Boltzmann Distribution

Before "how to optimize numerically," look at the objective's **closed-form optimum** — it shows why we must fall back to sampling. For fixed x , maximize $\mathbb{E}_{y \sim \pi} [s] - \beta \text{KL}(\pi \| \pi_{\text{ref}})$ under the normalization constraint, form the Lagrangian, take the functional derivative w.r.t. $\pi(y)$, set it to 0, and solve:

$$\pi^*(y | x) = \frac{1}{Z(x)} \pi_{\text{ref}}(y | x) \exp\left(\frac{1}{\beta} s(x, y)\right), \quad Z(x) = \sum_y \pi_{\text{ref}}(y | x) e^{s(x, y)/\beta}$$

This is the **Boltzmann / Gibbs distribution** — base π_{ref} reweighted exponentially by the score s , with temperature β . **Isomorphic to the maximum-entropy model in physics**.

Why you can't just use this closed form: $Z(x) = \sum_y \pi_{\text{ref}}(y) e^{s/\beta}$ sums over **all possible outputs** y — for an LLM, y ranges over all possible sentences, an **exponential** count in vocabulary size. **$Z(x)$ is not computable**. So π^* has an analytic form but no computable value — you can't use it as a backprop target.

Three routes around Z: (1) **MCMC sampling** (needs no Z, but parameters don't move; you must re-run to use it); (2) **policy gradient + sampled gradient estimate (PPO takes this route)**; (3) **cancel Z algebraically** (in a pairwise subtraction Z cancels $\rightarrow$ DPO, back to $y^+ - y^-$, no RL).

6 Core: How Policy Gradient Is Actually Solved (the part after the cost)

The focus of this document. Objective: $J(\theta) = \mathbb{E}_{y \sim \pi_\theta}[R(x, y)]$, where $R = s(x, y) - \beta \log \frac{\pi_\theta(y|x)}{\pi_{\text{ref}}(y|x)}$.

First, Nail Down What $\pi_\theta(y)$ Is

$\pi_\theta(y | x)$ is **not** "feed in y, get a score." It's **the probability of generating the whole string y**:

$$\pi_\theta(y | x) = \prod_{t=1}^L \pi_\theta(y_t | x, y_{<t})$$

This is the **chain rule** (each token conditioned on the prefix), **not an independence assumption**. The product form looks like independence, but each factor carrying $|y_{<t}$ means it isn't.

θ is a **network parameter (not a latent variable)**; it defines the distribution over y explicitly and differentially. **The probability of a single y, $\pi_\theta(y)$, is closed-form, computable and differentiable** (product of the selected-token softmax probabilities); **what has no closed form is the global normalizer Z and the sampling operation**. Policy gradient uses only the computable part.

Deriving the Gradient: the log-derivative trick

Write as a sum, apply the product rule: $\nabla_\theta J = \sum_y \nabla_\theta \pi_\theta(y) R(y) + \sum_y \pi_\theta(y) \nabla_\theta R(y)$.

Term ① hits a snag: $\nabla_\theta \pi_\theta(y)$ is not an expectation, can't be sampled. Use the **log-derivative trick** $\nabla_\theta \pi_\theta(y) = \pi_\theta(y) \nabla_\theta \log \pi_\theta(y)$:

$$\text{①} = \mathbb{E}_{y \sim \pi_\theta}[R(y) \nabla_\theta \log \pi_\theta(y)]$$

Term ②: $\nabla_\theta R = -\beta \nabla_\theta \log \pi_\theta$, and $\mathbb{E}_{\pi_\theta}[\nabla_\theta \log \pi_\theta] = \nabla_\theta \sum_y \pi_\theta = \nabla_\theta 1 = 0$, so its **expectation is zero**.

$$\nabla_\theta J = \mathbb{E}_{y \sim \pi_\theta}[R(x, y) \nabla_\theta \log \pi_\theta(y)] \approx \frac{1}{N} \sum_{i=1}^N R(x, y^{(i)}) \nabla_\theta \log \pi_\theta(y^{(i)}), \quad y^{(i)} \sim \pi_\theta$$

Key Clarification: y Is a Constant, You Don't Differentiate It

In $J = \sum_y \pi_\theta(y) R(y)$, y is the **summation dummy index** (ranging over all sentences), **not the function** $y(\theta)$. For each fixed y , $R(y)$ is a **definite number not containing θ** , so $\nabla_\theta R(y) = 0$ holds **exactly, not approximately**.

That " θ influences which y is sampled" equals exactly " θ influences each y 's **probability** $\pi_\theta(y)$," and that influence is **fully** captured by Term ①. $\nabla_\theta y$ **never appears** — it wasn't made to vanish; it should never have been written.

On "**discrete is non-differentiable**": discrete-sampling gradients **can be computed, with several unbiased estimators** (score function/REINFORCE, Gumbel-Softmax, measure-valued, etc.). RL uses the score-function estimator — valid for discrete, needing only $\nabla_\theta \log \pi_\theta(y)$ differentiable (log-probability always is), **bypassing $\nabla_\theta y$** . So it's not "discrete can't be differentiated"; policy gradient **chooses** not to pass through the sampling, treating reward as an external constant and differentiating only the probability π_θ .

Intuition: reward-weighted maximum likelihood

$\nabla_\theta \log \pi_\theta(y)$ is the **parameter direction** that makes the model more likely to generate y (a vector, not a scalar). Multiplied by the advantage A : $A > 0$ (good answer) weights positively, **pushes up** its probability; $A < 0$ (bad answer) weights negatively, **pushes down**. So policy gradient is **reward-weighted maximum likelihood**.

7 From Raw Gradient to PPO: Three Patches

Patch 1: Subtract a Baseline (reduce variance)

In the raw gradient R is an absolute score; if all R hover around 100, multiplying by $\nabla_\theta \log \pi_\theta$ drowns the signal in noise. Subtract a baseline $b(x)$ that is **independent of y** ; the gradient's expectation is unchanged ($E[b \cdot \nabla_\theta \log \pi_\theta] = b \cdot 0 = 0$). Take $b = V(x)$ to get the **advantage** $A(x, y) = R(x, y) - V(x)$.

A precise point (often taught wrong): subtracting a constant baseline **does not change the variance of R itself** (a constant shift doesn't change variance). What actually drops is the variance of the **product $R \cdot \nabla_\theta \log \pi_\theta$** — because the coefficient's absolute magnitude shrinks from ~ 100 to ~ 0 , its amplification of $\nabla_\theta \log \pi_\theta$'s fluctuation shrinks. The optimal baseline is in fact $\frac{E[(\nabla \log \pi)^2 R]}{E[(\nabla \log \pi)^2]}$, of which $V(x)$ is a handy approximation. Variance reduction works **through interaction with $\nabla \log \pi$** , not by shrinking R alone.

Who supplies $V(x)$? **Train another network (critic)** $\rightarrow$ actor + critic = **actor-critic**. Or (GRPO) sample a group of answers for the same x and use the **group-mean score** as $V(x)$, training no critic.

Patch 2: Importance Sampling (reuse old samples)

Sampling is the most expensive step; using a batch for only one update is wasteful. To let a batch drive **several updates**, but every update changes θ so the samples "expire" (they came from the old π_{old}). Correct with the probability ratio $\rho = \pi_{\theta}(y) / \pi_{\text{old}}(y)$: $J \approx E_{y \sim \pi_{\text{old}}}[\rho A]$.

Importance sampling is an exact identity, not an approximation:

$$E_{y \sim \pi_{\text{old}}} \left[\frac{\pi_{\theta}(y)}{\pi_{\text{old}}(y)} f(y) \right] = E_{y \sim \pi_{\theta}} [f(y)]$$

Multiplying by ρ **exactly switches** the expectation's base from π_{old} to π_{θ} . Estimating the new gradient from old samples + ρ is **unbiased in expectation**.

What ρ corrects: it does **not** mix old gradients into the new (it's not momentum/smoothing); it **computes a brand-new gradient from the old samples**, with ρ correcting the "samples came from the old distribution" bias. Like a **currency conversion**: old samples priced in "old currency" (π_{old}), ρ converts them to "new currency" (π_{θ}) pricing; the result is a pure new gradient, no old gradient in it.

Without ρ : at the second update θ has changed; without ρ you compute $E_{y \sim \pi_{\text{old}}} [A \nabla \log \pi_{\theta_1}]$, whose base is the **wrong distribution** (should be π_{θ_1}) $\rightarrow$ **a biased gradient**. Manifested as: over-pushing already-raised good samples and over-suppressing already-lowered bad ones, overfitting this batch and possibly collapsing after a few rounds.

Patch 3: Clip (clamp to prevent blowup)

ρ has a flaw: if π_{θ} walks too far, ρ becomes huge/tiny, IS variance blows up, a single sample dominates the gradient, training collapses. Clamp ρ to $[1 - \epsilon, 1 + \epsilon]$:

$$L^{\text{PPO}} = E[\min(\rho A, \text{clip}(\rho, 1 - \epsilon, 1 + \epsilon) A)]$$

Clip clamps ρ (the probability ratio), not A (the advantage). This distinction is core: $A = R - V$ is the signal itself (how much better than average this answer is), **which must not be limited**; ρ is the update step size, **which is limited** so no single sample's contribution runs away (a huge ρ lets one sample hijack the whole gradient). So clip ensures **no single training sample's gradient contribution is too large**, achieved by clamping ρ and keeping A .

The Three-Piece Set at a Glance

Piece	Job	Varies with
advantage A	sets direction (push good, suppress bad) and magnitude	the signal itself, unchanged
$\rho = \pi_{\theta} / \pi_{\text{old}}$	corrects the weight from old to new distribution (unbiased)	recomputed at each new θ
clip	clamps, prevents single-sample blowup	clamps ρ to $[1 - \epsilon, 1 + \epsilon]$

$\nabla_{\theta} \log \pi_{\theta}$ always follows the latest θ ; ρ only fixes each sample's weight, never touching that direction.

8 A Full Numerical Example: Walking Through One PPO Update

Sampling phase (with π_{old} , expensive, done once)

Let $M = 2$ prompts, each sampling $N = 4$ answers, 8 samples total. Scores: $x_1 : R = [2, 5, 1, 4]$, $x_2 : R = [3, 3, 6, 0]$. Record each sample's probability under π_{old} .

Advantage (group-mean baseline)

$\bar{R}_1 = 3 \Rightarrow A = [-1, +2, -2, +1]$; $\bar{R}_2 = 3 \Rightarrow A = [0, 0, +3, -3]$. Better-than-group-average $A > 0$ (push up), worse $A < 0$ (push down).

Update phase

Update 1 ($\theta = \theta_{\text{old}}$, every $\rho = 1$): $g_1 = \frac{1}{8} \sum_i A_i \nabla_{\theta} \log \pi(y_i)|_{\theta_{\text{old}}}$. No clip triggered. Step $\rightarrow \theta_1$ (y_2 pushed up, y_8 pushed down).

Update 2 ($\theta = \theta_1$, same 8 samples): re-forward for new probabilities, $\rho_i = \pi_{\theta_1}(y_i) / \pi_{\text{old}}(y_i)$ no longer 1. E.g. y_2 (pushed up) $\rightarrow \rho_2 = 1.3$; y_8 (pushed down) $\rightarrow \rho_8 = 0.6$. $\rho_2 = 1.3 > 1.2 \Rightarrow$ **clip triggers**, clamped to 1.2 — meaning y_2 is pushed enough, don't push more. Step $\rightarrow \theta_2$. Updates 3, 4 similarly. Once squeezed dry, return to sampling and re-sample.

"How many updates per batch" is a hyperparameter (epochs per rollout, typically 2–4), set by hand. More updates $\rightarrow \theta$ further from $\pi_{\text{old}} \rightarrow$ more samples' ρ go out of bounds and get clipped $\rightarrow$ **diminishing returns**. **Note:** not-using- $\rho \neq$ "force-update with $\rho=1$ " (the latter is a biased wrong move); it means "you can only re-sample every step and update once."

How to compute $\pi_{\theta_1}(y)$: y fixed, re-forward to look up probabilities

y is fixed (e.g. [cat, is, cute]), forward with the new θ_1 , look up each token's softmax probability and multiply: $\pi_{\theta_1}(y) = 0.5 \times 0.4 \times 0.6 = 0.12$. Under old θ_{old} the product is 0.10, so $\rho = 0.12 / 0.10 = 1.2$.

Why this step is cheap: computing $\pi_{\theta_1}(y)$ is **one parallel forward pass** (all of y is known, all positions fed at once — teacher forcing). Whereas **sampling** is autoregressive, token-by-token serial, much slower. So it's preferable to recompute old samples' new probabilities with ρ than to re-sample.

9 Serial/Parallel • Generation • Value • General RL • Networks

Serial vs Parallel: One Model, Two Timings

Why must generation be serial while computing probability can be parallel? One sentence: **does the next step's input have to wait for the previous step's output?**

	What you have	Where the next input comes from	So
Sampling (generate)	only x , tokens made on the fly	depends on the token just drawn	must wait → serial
Prob. (evaluate)	the whole sentence is known	taken directly from the known sentence	no waiting → parallel

Parallel $\neq$ independent: parallel only means "since y is known, all positions compute at once"; each position still sees only its prefix ($< t$) via causal attention. The conditional dependence is fully intact, and probabilities from both timings are **identical**.

Generation: How x Produces y Step by Step

Not "attend over x once and done," but **each token generated is appended and attention re-run**: sequence = $[x]$ → causal self-attention → take the **last position** vector h_n → $h_n \times W_{\text{unembed}}$ → logits → softmax → draw token = y_1 → append, sequence = $[x, y_1]$ → **re-run attention over the whole sequence** → draw y_2 → ... until EOS.

The attended sequence **grows**: step t acts on $[x, y_1, \dots, y_{t-1}]$. x **always sits at the front as the anchor**, so every generated token attends to the prompt and all its own preceding tokens — this is why answers stay on-topic and coherent. (**KV cache** stores earlier tokens' K/V to avoid recomputation — a speedup, not a logic change.)

Noise, Chaos, and Error Accumulation

Autoregression carries the seed of chaos (recursion + nonlinearity), but several mechanisms hold it down: the prompt x as a fixed anchor, language as a strong-constraint attractor (grammar/semantics auto-correct), softmax normalizing each step against numerical drift, and training pushing toward a self-consistent low-entropy region. So noise is **bounded**, not exponentially divergent; but when unheld (high temperature, beyond training length, adversarial prompts, repetition degeneration) it **does** drift away.

The train-stable / generate-drifting asymmetry (exposure bias): training uses teacher forcing, feeding the **true prefix** each step, corrected immediately when wrong; at generation the prefix is **self-produced**, and once it drifts nobody pulls it back. **Standard cross-entropy doesn't manage this accumulation** — which is exactly **one fundamental motivation for RLHF**: cross-entropy handles per-token only, not the whole sequence; RL's sequence-level reward scores the **entire generation** and naturally penalizes accumulated collapse.

A philosophical note: attention's structure (each token sees the full prefix) aligns with the chain decomposition of language probability $\prod_t P(y_t|y_{<t})$ — the root of why it beats n-grams/RNNs. But precisely: **structural alignment $\neq$ being the true distribution**; it is a learnable approximator with error. Deeper still: **whether language itself is a probabilistic process is itself in question** — probabilistic modeling is a lens we **choose** to place on language, not language's intrinsic nature.

Value Function: Where the Definition Comes From

$V^\pi(s) = E_\pi[\sum_{t=0}^{\infty} \gamma^t r_t \mid s_0 = s]$. Three design decisions: **future accumulation** (want long-term good, not immediate), **discount** γ (infinite sum must converge, far future is less reliable, earlier is worth more), **expectation** (the future is random, so average into an optimizable scalar).

Ultimate value: it satisfies the **Bellman recursion** $V(s) = E[r + \gamma V(s')]$. With it you needn't simulate to infinity — you can solve iteratively (DP, TD). **The definition is mathematically essential (not a compute-compromise); but "how to estimate value" has many methods (MC/TD/n-step/GAE/DP/neural approximation), and the trade-offs among them are what get forced by compute, memory, data, and whether the model is known.**

General RL: Reading the Multi-Step Version Against RLHF

RLHF is the **single-step collapse** of general PPO under "the whole response = one action, only one reward at the end." The three-piece set is identical on both sides; you only expand each piece from "one" to "one per timestep," and add GAE to estimate multi-step advantage.

Piece	RLHF (single step)	General RL (multi-step)
Interaction	one question, one answer	multi-step trajectory $\tau = (s_0, a_0, r_0, \dots)$
Action	whole response y as one action	one a_t per step
Reward	one $R(x, y)$ at the end only	r_t each step
Objective J	$E_y[R(x, y)]$	$E_\tau[\sum_t \gamma^t r_t]$
Value	$V(x) =$ answer average (degenerate)	$V(s_t) =$ multi-step discounted sum
Advantage	$A = R - V(x)$, one	$A_t = Q(s_t, a_t) - V(s_t)$, per step
Estimate A	direct subtraction	GAE (accumulate over future)
ρ	one for the whole response	ρ_t per step
clip	clip one ρ	clip each ρ_t

General objective and gradient: $J(\theta) = E_{\tau \sim \pi_\theta}[\sum_{t=0}^T \gamma^t r_t]$, $\nabla_\theta J = E[\sum_t \nabla_\theta \log \pi_\theta(a_t|s_t) \cdot A_t]$.

t is the future, not the past. J is an expectation over **not-yet-happened trajectories sampled by π_θ** . Using "already-run trajectories" to estimate it at training time is merely the **sampling-estimation mechanism** — strictly, what's estimated is $\nabla_\theta J$. **The future is estimated by actually running / self-play sampling; without running, there's nothing.**

GAE: $\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$ (single-step TD residual), $A_t^{\text{GAE}} = \sum_{l=0}^{\infty} (\gamma \lambda)^l \delta_{t+l}$. λ tunes bias-variance: $\lambda = 0$ all value estimate (biased, stable), $\lambda = 1$ all actual return (unbiased, high variance). These all degenerate in single-step RLHF.

In one line: general PPO = expanding RLHF's "single step" along the time axis into "multi-step." The three-piece set is identical; the only real addition is that value goes from "answer average" back to "multi-step discounted sum," and advantage goes from "one subtraction" to "GAE." **Once you've mastered RLHF, general RL has no new core mechanism to learn.**

How Many Networks in RLHF (closing summary)

Network	What it is	Trained?
policy π_θ	the LLM itself being trained (final product)	yes
reference π_{ref}	frozen copy of the SFT base, only for KL	no
reward model s	scorer (pairwise-trained), as judge	no (frozen)
value / critic V_ϕ	estimates the baseline (GRPO uses group mean instead)	yes (not final product)

The policy network is that LLM. policy and reference are "two versions of the same LLM" — reference is the frozen snapshot of the SFT model at the start; the policy departs from it and is continually updated by RL. The KL term $\log \frac{\pi_\theta}{\pi_{\text{ref}}}$ measures "how far the ever-training LLM has drifted from its starting self." The rest are auxiliaries.

End of Part 1 — from reward diagnosis to PPO solving. The core: the cost function is only the start; "Z uncomputable → fall back to sampling → log-trick yields the gradient → baseline / ρ / clip three patches" is the real content of PPO.